



```
In [1]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [2]: # Autoencoder for Image Denoising using PyTorch
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
from torchsummary import summary
```

```
In [3]: # Device configuration
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
In [4]: # Transform: Normalize and convert to tensor
```

```
transform = transforms.Compose([
    transforms.ToTensor()
])
```

```
In [5]: # Load MNIST dataset
```

```
dataset = datasets.MNIST(root='./data', train=True, download=True, transform=t
test_dataset = datasets.MNIST(root='./data', train=False, download=True, trans

train_loader = DataLoader(dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 17.9MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 492kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.50MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 11.3MB/s]
```

```
In [6]: # Add noise to images
```

```
def add_noise(inputs, noise_factor=0.5):
    noisy = inputs + noise_factor * torch.randn_like(inputs)
    return torch.clamp(noisy, 0., 1.)
```

```
In [7]: # Denoising Autoencoder model
```

```
class DenoisingAutoencoder(nn.Module):
    def __init__(self):
        super(DenoisingAutoencoder, self).__init__()

        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1), # [B, 16, 1
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1), # [B, 32, 7
            nn.ReLU()
        )
```

```

        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1, out
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1, outp
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x

```

```

In [8]: model = DenoisingAutoencoder().to(device)
        criterion = nn.MSELoss()
        optimizer = optim.Adam(model.parameters(), lr=1e-3)

```

```

In [9]: # Print model summary
        summary(model, input_size=(1, 28, 28))

```

```

-----
Layer (type)          Output Shape          Param #
=====
      Conv2d-1         [-1, 16, 14, 14]         160
      ReLU-2           [-1, 16, 14, 14]           0
      Conv2d-3         [-1, 32, 7, 7]         4,640
      ReLU-4           [-1, 32, 7, 7]           0
  ConvTranspose2d-5     [-1, 16, 14, 14]         4,624
      ReLU-6           [-1, 16, 14, 14]           0
  ConvTranspose2d-7     [-1, 1, 28, 28]         145
      Sigmoid-8        [-1, 1, 28, 28]           0
=====
Total params: 9,569
Trainable params: 9,569
Non-trainable params: 0
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.13
Params size (MB): 0.04
Estimated Total Size (MB): 0.17
-----

```

```

In [10]: def train(model, loader, criterion, optimizer, epochs=5):
        model.train()

        for epoch in range(epochs):
            running_loss = 0.0

            for images, _ in loader:
                images = images.to(device)
                noisy_images = add_noise(images).to(device)

                # Forward pass
                outputs = model(noisy_images)

```

```

        loss = criterion(outputs, images)

        # Backward pass and optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch [{epoch+1}/{epochs}], Loss: {running_loss/len(loader):.4

```

```

In [13]: # Evaluate and visualize
def visualize_denoising(model, loader, num_images=10):
    model.eval()
    with torch.no_grad():
        for images, _ in loader:
            images = images.to(device)
            noisy_images = add_noise(images).to(device)
            outputs = model(noisy_images)
            break

    images = images.cpu().numpy()
    noisy_images = noisy_images.cpu().numpy()
    outputs = outputs.cpu().numpy()

    print("Name: CH.V.S.DINESH KUMAR ")
    print("Register Number: 212224040055 ")
    plt.figure(figsize=(18, 6))
    for i in range(num_images):
        # Original
        ax = plt.subplot(3, num_images, i + 1)
        plt.imshow(images[i].squeeze(), cmap='gray')
        ax.set_title("Original")
        plt.axis("off")

        # Noisy
        ax = plt.subplot(3, num_images, i + 1 + num_images)
        plt.imshow(noisy_images[i].squeeze(), cmap='gray')
        ax.set_title("Noisy")
        plt.axis("off")

        # Denoised
        ax = plt.subplot(3, num_images, i + 1 + 2 * num_images)
        plt.imshow(outputs[i].squeeze(), cmap='gray')
        ax.set_title("Denoised")
        plt.axis("off")

    plt.tight_layout()
    plt.show()

```

```

In [12]: # Run training and visualization
train(model, train_loader, criterion, optimizer, epochs=5)
visualize_denoising(model, test_loader)

```

Epoch [1/5], Loss: 0.0721
Epoch [2/5], Loss: 0.0154
Epoch [3/5], Loss: 0.0146
Epoch [4/5], Loss: 0.0141
Epoch [5/5], Loss: 0.0137
Name: CH.V.S.DINESH KUMAR
Register Number: 212224040055

